Annex F

Prototype Implementations and Experimental Setups for the RLA/CRC Framework

Gianluca Conte

Independent Researcher

November 23, 2025

Contents

1 Scope and Role of Annex F

This Annex provides a collection of *minimal, but concrete* prototype designs and experimental setups that instantiate the conceptual and formal structures introduced in the main paper and Annexes A–E. The goal is not to offer production–ready software, but to supply:

- reference architectures and pseudocode for core components (bryophyte simulator, ECNN, ECUs, UCEs, RLA–CA exploration);
- example experimental pipelines aligned with the Popper–χ protocol;
- guidance on reproducibility, logging and configuration.

The prototypes are deliberately lightweight: they aim to show that the constructs of RLA, CRC, ECNN, ECU and UCE are implementable with standard tools (e.g. Python, mainstream ML frameworks, basic numerical libraries), while preserving the epistemic and computability constraints assumed in the theoretical framework.

For the sake of clarity we assume a single reference repository, even though in practice the components could be split across multiple projects.

2 Reference Repository Structure

We assume a repository organised as follows (directory names are illustrative):

```
nexus-rla-crc/
  README.md
  requirements.txt
  config/
    bryophyte_default.yaml
    ecnn_mnist.yaml
    ecnn_cifar10.yaml
    ecu_text_triage.yaml
    uce_medical_triage.yaml
    rla_ca_experiments.yaml
```

```
src/
  common/
    logging_utils.py
    config_utils.py
    metrics.py
    popchi_harness.py
  rla_core/
    reticulum.py
    indices.py
    ioa_ec_tc_checks.py
  bryophyte/
    bryo_state.py
    bryo_update.py
    bryo_scenarios.py
    bryo_runner.py
  ecnn/
    ecnn_models.py
    ecnn_training.py
    ecnn_evaluation.py
  ecu/
    ecu_base.py
    ecu_llm_text.py
    ecu_epistemic_matrix.py
  uce/
    uce_base.py
    uce_medical_triage.py
  rla_ca/
    ca_rules.py
    rla_to_ca_mapping.py
    ca_metrics.py
experiments/
  bryophyte_scenarios/
  ecnn_benchmarks/
  ecu_challenges/
  uce_triage/
  rla_ca_sweeps/
logs/
results/
```

The following sections detail the conceptual and technical content of each group of modules and the associated experimental protocols.

3 Prototype F.1: Bryophyte CRC Simulator

3.1 Objectives

Prototype F.1 implements a compact reticulum for the bryophyte case study introduced in Annex B. The goals are:

F.1.1 to encode the multi–level structure (levels $L01$–$L20$, states, transmissions) into explicit data structures;

F.1.2 to implement a discrete daily loop T that is Turing–compatible (finite state, finite precision);

F.1.3 to support multiple climate/scenario configurations from the literature;

F.1.4 to log time series suitable for:

- epistemic indices (coefficient of collapse, index of emergence, reticular irreducibility);
- blind expert inspection for epistemic equivalence tests.

3.2 State Representation

We define a minimal state container for the compact reticulum:

- a fixed set of levels $\{L_1, \ldots, L_{20}\}$;
- for each level L_i, a finite–dimensional state vector $x_i(t) \in \mathbb{R}^{d_i}$;
- a global parameter vector θ collecting parameters $P01$–$P42$ as in Annex B (each with finite precision discretisation).

In Python-like pseudocode, the central state object may resemble:

```
class BryophyteState:
    def __init__(self, theta, init_levels):
        self.theta = theta          # parameters P01..P42
        self.levels = init_levels   # dict: level_id -> numpy array
        self.t = 0                  # current day index
```

3.3 Update Loop

The daily loop T is implemented as a finite sequence of updates:

1. update exogenous climate forcing and boundary conditions;
2. update water balance and internal moisture;
3. update stress and damage indicators (including ROS);
4. update epigenetic response and acclimation variables;
5. update demographic variables (growth, reproduction, mortality);
6. apply constraints, clipping and collapse events (if any);
7. log observer variables for analysis.

Each update function manipulates only a finite subset of levels and uses a bounded number of arithmetic operations on finite–precision floats, ensuring Turing–compatibility as required by the definition of compact reticula.

Algorithm 1 Prototype daily loop for the bryophyte CRC

Require: initial state S_0, parameters θ, forcing series $\mathcal{F}$, horizon $T_{\max}$

$S \leftarrow S_0$
for $t = 1$ to $T_{\max}$ **do**
  update_climate$(S, \mathcal{F}, t)$
  update_water_balance(S, θ)
  update_stress_and_damage(S, θ)
  update_epigenetics(S, θ)
  update_demography(S, θ)
  apply_constraints_and_collapse(S)
  log_observables(S)
  $S.t \leftarrow t$
end for

3.4 Scenario Configuration

Configuration files (e.g. YAML) specify:

- the climate forcing series (temperature, precipitation, radiation);
- the parameter vector θ (with defaults derived from Annex B);
- simulation horizon $T_{\max}$ (e.g. number of days, years);
- output variables to log and their sampling rate.

Example fragment:

```
scenario_name: "baseline_temperate"
horizon_days: 3650
climate_forcing:
  source: "synthetic"
  mean_temperature: 12.0
  seasonal_amplitude: 8.0
parameters:
  P01: 0.45   # field capacity threshold, ...
  P02: 0.20   # wilting point, ...
  ...
logging:
  observables:
    - "L12.vitality"
    - "L13.ros_index"
    - "L18.cover_fraction"
  frequency_days: 1
```

3.5 Outputs and Usage

For each run, the simulator writes:

- trajectory files (e.g. CSV, HDF5) with time series of selected observables;
- summary indices over the whole run (e.g. collapse events, resilience indicators);
- configuration snapshot (YAML) for reproducibility.

These artefacts are used by:

- epistemic indices computations (Annex A);
- blind expert inspection pipelines for epistemic equivalence tests (Annex B);
- alignment with Popper–χ challenge suites where the bryophyte simulator serves as an engineered CRC.

4 Prototype F.2: ECNN Benchmarks on Image Classification

4.1 Objectives

Prototype F.2 implements minimal ECNN architectures for standard image benchmarks (e.g. MNIST, CIFAR–10) in order to:

- verify the practicality of a separate epistemic channel L_{ep};
- test structured *unknown* and *contradiction* outputs;
- compare with baseline uncertainty/abstention methods under the same experimental protocol.

4.2 Architecture

We consider a base convolutional feature extractor f_θ and two heads:

- a *task head* h_ϕ producing class scores over K classes;
- an *epistemic head* g_ψ producing:
  - an epistemic status (normal / unknown / contradiction);
  - auxiliary signals (e.g. confidence score, entropy, novelty score).

  Formally, for an input image x:

$$z = f_\theta(x), \qquad \hat{y} = \arg\max_k h_\phi(z)_k, \qquad s_{\text{ep}} = g_\psi(z, h_\phi(z)).$$

  The epistemic head may use:

- thresholded confidence scores (e.g. max softmax probability);
- distance in latent space to training prototypes;
- a small auxiliary network trained to predict epistemic status labels.

4.3 Training Regime

We consider two training regimes:

Closed–world regime. Only in–distribution data; the epistemic head is trained (or calibrated) to approximate confidence, but *unknown* and *contradiction* are rarely used.

Extended regime. Additional datasets (e.g. rotated or corrupted digits, or an external dataset) act as OOD or adversarial cases and are labelled with epistemic statuses (unknown, sometimes contradiction for conflicting annotations).

Algorithm 2 ECNN training loop (extended regime)

Require: training data (x_i, y_i, e_i) where e_i is an epistemic label

Initialise parameters θ, ϕ, ψ
for epoch $= 1$ to $E_{\max}$ **do**
  for mini-batch (X, Y, E) **do**
    $Z \leftarrow f_\theta(X)$
    $L_{\text{task}} \leftarrow \text{cross\_entropy}(h_\phi(Z), Y)$
    $L_{\text{ep}} \leftarrow \text{epistemic\_loss}(g_\psi(Z, h_\phi(Z)), E)$
    $L \leftarrow L_{\text{task}} + \lambda L_{\text{ep}}$
    update (θ, ϕ, ψ) using a gradient–based optimiser
  end for
end for

4.4 Epistemic Decisions and Artefacts

At evaluation time, each input x yields an epistemic artefact

$$a = (y^*, q, s, R, \Pi)$$

where:

- y^* is either a class label, *unknown* or *contradiction*;
- q is a quantitative score (e.g. calibrated probability);
- s is a status flag (e.g. "OOD detected", "conflicting evidence");
- R is a short rationale (e.g. textual template instantiated from latent indicators);
- Π is a policy suggestion (e.g. *accept*, *escalate*, *discard*).

These structures align with the definitions of ECU artefacts in Annex E.

4.5 Metrics and Popper–χ Challenges

Prototype F.2 produces the following metrics:

- standard accuracy over in–distribution test data;
- unknown rate and contradiction rate on extended challenge suites;
- overconfidence rate: fraction of wrong predictions with high q but non–epistemic status;
- Popper–χ–oriented scores such as:
  - challenge pass rate (percentage of challenge cases where the epistemic decision matches the ground truth status);
  - fragility under perturbation (change in decision under small input perturbations).

All metrics and logs are produced via common utilities in `src/common/metrics.py` and `src/common/popchi_harness.py`.

5 Prototype F.3: Textual ECUs as Epistemic Oracles

5.1 Objectives

Prototype F.3 implements ECUs that operate on textual inputs using large language models (LLMs), under a regime that remains practically deterministic and Turing–compatible. The goals are:

- to instantiate the notion of epistemic matrix M and epistemic artefacts on natural–language tasks;
- to show how ECUs can be used as pluggable epistemic oracles in larger UCEs;
- to support Popper–χ challenge suites with contradictory pairs, adversarial prompts and epistemic triggers.

5.2 Basic ECU Interface

Each ECU is exposed as a pure function

$$\mathrm{ECU}_{\varphi} : Z \times M \longrightarrow A$$

where:

- Z is a finite set of input features (prompt, context, additional flags);
- M is the epistemic matrix specifying rules, constraints, and policies;
- A is the set of epistemic artefacts as in Prototype F.2.

In pseudocode:

```
class ECUText:
    def __init__(self, model, epistemic_matrix, decoder_config):
        self.model = model                # fixed LLM checkpoint
        self.M = epistemic_matrix         # rules, constraints, policies
        self.dec_cfg = decoder_config     # deterministic decoding

    def __call__(self, z):
        prompt = self._build_prompt(z)
        raw_output = self._query_model(prompt)
        parsed_output = self._parse_output(raw_output)
        artifact = self._apply_epistemic_matrix(parsed_output)
        return artifact
```

5.3 Deterministic Regime

To approximate deterministic behaviour:

- the LLM checkpoint is fixed and versioned;
- decoding parameters enforce near–determinism (e.g. temperature 0, or constrained top–k);
- prompt templates and output parsers are fully specified and tested;
- the epistemic matrix M is a finite, explicitly encoded object.

No non–computable oracle is assumed: the model is treated as a large but finite program with fixed parameters.

5.4 Prompt Templates and Parsing

A generic template:

```
[INSTRUCTIONS]
You are an epistemic diagnostic unit (ECU) for task <TASK_NAME>.
You must answer using STRICT JSON with the following keys:
  "answer": <string>,
  "status": <"normal" | "unknown" | "contradiction">,
  "confidence": <number in [0,1]>,
  "rationale": <short text>,
  "policy": <"accept" | "escalate" | "reject">.

[CONTEXT]
<optional background or case description>

[QUESTION]
<Q>
```

The parser checks JSON validity and enforces type and range constraints before passing the data to the epistemic matrix M.

5.5 Epistemic Matrix and Policies

The epistemic matrix M encodes:

- allowable combinations of (answer, status, confidence);
- consistency rules between different ECUs (when used in a UCE);
- abstract policies (when to escalate, when to accept, when to reject).

For example, M might enforce:

- if `status = "unknown"` then `policy` is either `"escalate"` or `"reject"`;
- if `confidence < 0.3` and `status = "normal"` then flip status to *unknown*;
- if answers from two ECUs disagree with high confidence then set joint status to *contradiction*.

5.6 Challenge Suites

Prototype F.3 includes:

- *normal* cases with a relatively clear ground truth;
- *hard* or ambiguous cases where the correct epistemic output is *unknown*;
- *contradictory* pairs, where two authoritative sources disagree;
- adversarial prompts designed to trigger over–confident but wrong answers.

For each case, we track artefacts and evaluate metrics as defined in Annex E (unknown rate, contradiction rate, overconfidence, coherence of policies).

6 Prototype F.4: Epistemic Use Case Environment (UCE)

6.1 Objectives

Prototype F.4 instantiates a UCE for a simplified safety–critical use case (e.g. medical triage, regulatory screening or compliance assessment). The goals are:

- to show how multiple ECUs can be orchestrated;
- to demonstrate epistemic conflict resolution and escalation policies;
- to provide a concrete ground for Popper–χ challenge suites at the UCE level.

6.2 UCE Composition

A UCE bundles:

- a set of ECUs $\{\mathrm{ECU}_1, \ldots, \mathrm{ECU}_n\}$, each with its own epistemic matrix M_i;
- a coordination matrix M_{coord} that defines how artefacts are combined;
- a logging layer for all inputs, intermediate artefacts and final decisions.

In a medical triage example, we might use:

- ECU_1: symptom classifier (normal vs urgent);
- ECU_2: risk factor assessor;
- ECU_3: guideline checker.

6.3 Orchestration Algorithm

Algorithm 3 Basic orchestration loop for a UCE

Require: case description z, ECUs $\{\mathrm{ECU}_i\}_{i=1}^{n}$, coordination matrix M_{coord}

$A \leftarrow []$ {list of artefacts}
for $i = 1$ to n **do**
  $a_i \leftarrow \mathrm{ECU}_i(z)$
  append a_i to A
end for
$a_{\mathrm{final}} \leftarrow \mathrm{combine\_artefacts}(A, M_{\mathrm{coord}})$
log $(z, A, a_{\mathrm{final}})$
return a_{final}

The combination step applies M_{coord}:

- if all ECUs agree on a benign decision with high confidence, output *accept*;
- if at least one ECU signals *unknown* or *contradiction*, or if there is significant disagreement, output *escalate*;
- in rare cases of strong conflict, output *reject* with a summary rationale.

6.4 UCE–level Metrics

In addition to ECU–level metrics, we compute:

- disagreement rate among ECUs;
- escalation rate and its calibration (how often escalation was appropriate);
- coverage (fraction of cases resolved without escalation);
- resilience under Popper–χ challenges (robustness to adversarial and ambiguous cases).

7 Prototype F.5: RLA–CA Exploration Environment

7.1 Objectives

Prototype F.5 supports exploratory experiments linking compact reticula and one–dimensional cellular automata, as discussed in Annex D. The goals are:

- to instantiate simple maps $\Phi : CRC \to CA$ and $\Psi : CA \to CRC$ at least for small reticula;
- to explore empirical relationships between indices (collapse, emergence, irreducibility) and CA behaviours;
- to provide concrete data for or against the speculative conjectures about RLA, CRC and the Principle of Computational Equivalence.

7.2 Encoding and Mapping

We restrict to:

- small reticula with a handful of levels and binary or few–valued states;
- elementary cellular automata (radius 1, binary states, rules 0–255).

  A simple mapping Φ may:

- encode the structure of transmissions and their injectivity/non–injectivity into a pattern of local update rules;
- use indices like the coefficient of collapse and reticular irreducibility index to bias the selection of CA rules;
- restrict to a subset of CA rules known to exhibit different Wolfram classes (I–IV).

7.3 CA Simulation and Metrics

The environment provides:

- a CA simulator capable of evolving a configuration over a finite time horizon;
- metrics such as:
  - empirical entropy of space–time diagrams;
  - activity measures (density of state changes);
  - simple Lyapunov–like indicators or damage–spreading metrics.

Each run logs:

- the originating reticulum (structural encoding, indices);
- the chosen CA rule(s);
- the measured CA metrics over multiple initial conditions.

7.4 Exploratory Protocols

Prototypical experiments include:

- sweep over reticula: random generation of small compact reticula with varying collapse and irreducibility, mapping to CA rules, and aggregation of CA metrics by reticular indices;
- sweep over CA rules: sampling rules, computing CA metrics, and inferring plausible reticular indices via Ψ.

The results are not intended as definitive evidence, but as systematic exploratory data to inform the conjectures of Annex D.

8 Prototype F.6: Generic Popper–χ Harness

8.1 Objectives

Prototype F.6 provides a generic harness for Popper–χ challenge suites targeting:

- single ECUs;
- ECNNs;
- UCEs (composite environments).

8.2 Challenge Suite Representation

A challenge suite is represented as:

- a set of cases, each with:
  - an input description z;
  - a ground truth task label (when applicable);
  - a ground truth epistemic status (normal/unknown/contradiction);
  - metadata such as difficulty and provenance;
- a configuration specifying:
  - which system(s) to test (single ECU, ECNN, UCE);
  - which metrics to compute;
  - stopping criteria or resource limits.

Algorithm 4 Popper–χ evaluation harness (simplified)

Require: system S, challenge suite $\mathcal{C}$

initialise counters for metrics
for case c in $\mathcal{C}$ **do**
    $z \leftarrow c$.input
    $a \leftarrow S(z)$
    update_metrics(a, c)
    log (c, a)
end for
compute final scores and summaries
return metrics, logs

8.3 Evaluation Loop

To remain consistent with the epistemic framework, the harness:

- extracts y^*, q, s, R, Π from each artefact;
- compares them with task and epistemic ground truths;
- updates the relevant metrics (unknown rate, contradiction rate, overconfidence rate, fragility, etc.).

8.4 Reporting

The harness produces:

- summary tables for all metrics;
- confusion matrices extended with epistemic statuses;
- qualitative samples of interesting failures and successes (with rationales R and policies Π).

These reports are meant to support peer review and critical examination of the epistemic behaviour of the systems, more than to optimise performance.

9 Reproducibility and Extension Guidelines

9.1 Configuration and Logging

All prototypes follow a common pattern:

- configuration files in human–readable formats (e.g. YAML) fully specify:
  - model architectures and hyperparameters;
  - datasets and preprocessing;
  - epistemic matrices and policies;
  - random seeds and resource limits;
- structured logs (e.g. JSONL, CSV) capture:
  - inputs, outputs and artefacts;
  - intermediate diagnostics;
  - environment information (software versions, hardware identifiers).

9.2 Deterministic Regime and Turing Compatibility

To honour the assumptions of compact reticula and deterministic ECUs:

- all sources of randomness (initialisation, data shuffling, decoding) are seeded;
- LLM–based ECUs use fixed checkpoints and decoding parameters;
- numerical precision and discretisation are kept finite and documented;
- no external oracle is invoked beyond the specified software artefacts.

9.3 Extending the Prototypes

Reviewers and practitioners are encouraged to:

- plug in alternative architectures (e.g. transformers instead of CNNs) as long as they comply with the same epistemic interfaces and reticular constraints;
- design additional Popper–χ challenge suites, especially for domains where epistemic failure is critical;
- explore richer mappings between RLA, CRC and CA, beyond the simple schemes of Prototype F.5.

This Annex is intentionally *minimal yet varied*: each prototype can be implemented with modest effort, but together they cover the main axes of the Nexus framework—natural CRCs, engineered CRCs, epistemic units and environments, and their relation to computability and complexity—thus supporting a thorough and critical peer review.